

Reviewer Pack — Minimal Lattice Width in Integer Lattices vs Monotone Circui...

Entry 7934dd5b4fc3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 11:45:40 UTC

Minimal Lattice Width in Integer Lattices vs Monotone Circuit Lower Bounds for k-CLIQUE

Entry ID: 7934dd5b4fc3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 11:45:40 UTC

1. Conjecture

Field A (mathematical branch): Integer Lattices **Field B** (complexity object): Monotone Circuit Complexity for k-CLIQUE

Statement:

[‘For every instance of the k-CLIQUE problem, there exists a representation as an integer lattice with width at most $n^{\{3/4\}} \log^2(k)$, where n is the number of vertices.’, ‘The lattice width of this representation determines a monotone circuit lower bound for k-CLIQUE, such that any monotone circuit solving k-CLIQUE must have size at least $\Omega(n^{\{5/4\}})$.’]

Rationale (proposer’s reasoning):

[‘Integer lattices provide a structured way to represent the combinatorial properties of graphs. The width of an integer lattice is related to the complexity of computations that can be performed on it, and thus could potentially give insight into circuit complexity.’, ‘This conjecture connects the geometric structure of integer lattices with the computational hardness of k-CLIQUE, which has implications for monotone circuit lower bounds.’]

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 340732345000f575

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

If for all tested k-CLIQUE instances with $n \leq 40$, the lattice width is at most $n^{\{3/4\}} \log^2(k)$ and the monotone circuit size is at least $\Omega(n^{\{5/4\}})$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	UNCERTAIN	0.90	HITS	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'integer lattices' AND 'monotone circuit complexity' AND 'k-CLIQUE' - 'lattice width' AND ' $\Omega(n^{\{5/4\}})$ ' AND 'monotone circuits' - 'k-CLIQUE problem' AND ' $n^{\{3/4\}} \log^2(k)$ ' AND 'integer lattices'

Top relevant hits considered: - [http://arxiv.org/abs/1006.0686v1] Integer Sequences from Queueing Theory - [http://arxiv.org/abs/1503.01218v2] Maximizing Monotone Submodular Functions over the Integer Lattice - [http://arxiv.org/abs/1504.00389v1] Some Elementary Congruences for the Number of Weighted Integer Compositions - [http://arxiv.org/abs/1308.1705v1] Rotating lattice - [http://arxiv.org/abs/1204.5424v1] Hybrid Monte Carlo simulation on the graphene hexagonal lattice - [http://arxiv.org/abs/1210.7794v1] Glueball spectral densities from the lattice - [http://arxiv.org/abs/1812.11098v1] Isolation of k -cliques - [http://arxiv.org/abs/2112.03983v3] Almost Polynomial Factor Inapproximability for Parameterized k -Clique

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        pivot = A[i][i]
        if pivot == 0:
            continue
        for j in range(i+1, n):
            factor = -A[j][i] / pivot
            for k in range(n+1):
                A[j][k] += factor * A[i][k]
    rank = sum(1 for row in A if any(row))
    return rank

def lattice_width(A, B):
    n = len(A)
    rank = gaussian_elimination(A)
```

```

width = 2 ** (n - rank)
return width

def construct_lattice(G):
    n = len(G)
    A = [[0] * (n + 1) for _ in range(n)]
    B = [[0] * (n + 1) for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            if G[i][j]:
                A[i][j] = 1
                A[j][i] = -1
                B[i][j] = 1
                B[j][i] = -1
    return A, B

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        k = random.randint(2, min(n-1, 10))
        G = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
        for i in range(n):
            G[i][i] = 0
        A, B = construct_lattice(G)

        width = lattice_width(A, B)
        expected_width = n ** (3/4) * math.log(k) ** 2

        if width > expected_width:
            return {
                "metric_name": "Lattice Width",
                "metric_value": width,
                "instances_tested": len(n_values),
                "conjecture_holds": False,
                "counterexample": f"n={n}, k={k}, lattice_width={width} > {expected_width}"
            }

    mean_width = sum(result["metric_value"] for result in results) / len(results)
    std_width = math.sqrt(sum((result["metric_value"] - mean_width) ** 2 for result in results) /
                           len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    return {
        "metric_name": "Lattice Width",
        "metric_value": mean_width,
        "instances_tested": len(n_values),
        "conjecture_holds": support_fraction >= 0.8,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

```

```

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_width = sum(result["metric_value"] for result in results) / len(results)
std_width = math.sqrt(sum((result["metric_value"] - mean_width) ** 2 for result in results) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_width} std={std_width} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"first failing seed\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_50530b2b.py", line 97, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_50530b2b.py", line 79, in run_trial
    mean_width = sum(result["metric_value"] for result in results) / len(results)
                  ~~~~~^
ZeroDivisionError: division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test without errors to verify if the conjecture holds for all tested k-CLIQUE instances with $n \leq 40$.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14795
2	propose	ollama_remote	glm4:latest	0	0	15524
3	preregistration	ollama_remote	glm4:latest	0	0	9378
4	novelty	ollama_remote	glm4:latest	0	0	8849
5	novelty	ollama_remote	glm4:latest	0	0	9533
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14525
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12080
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11868
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11778
10	judge	ollama_remote	glm4:latest	0	0	11791

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 120119 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/7934dd5b4fc3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7934dd5b4fc3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/7934dd5b4fc3.tar.gz` (if generated)